

Hands-on Lab: Windows Firewall with Advanced Security

This lab is part of a TryHackMe practice room. In this exercise, I demonstrated how to secure a Windows Operating System using the host-based protections provided by Windows Defender Firewall. I achieved this through two primary phases:

1. Configuring basic application rules using the standard Windows Defender Firewall interface.
2. Building security policies using Windows Defender Firewall with Advanced Security (WDFAS).

I mapped out and executed four distinct scenarios to control various aspects of inbound and outbound network traffic:

- Scenario **A**: Blocked Remote Desktop Protocol (RDP) on the Public Network (Inbound Rule)
- Scenario **B**: Blocked Outbound Traffic for Specified Applications (Outbound Rule)
- Scenario **C**: Blocked Web Server (HTTP) Traffic on a Public Network (Inbound Rule)
- Scenario **D**: Allowed Key Management Service on the Domain and Private Network and denied the connection on the Public Network (Inbound Rule)

Phase 1: Configuring basic application rules with the standard Windows Defender Firewall interface

Step 1: Network Profile Auditing

Before configuring the specific scenarios, I **verified** the current Firewall status across different network profiles. I navigated to **Start Menu > Windows Security > Firewall & network protection** to analyse the three distinct Windows network profiles:

- **Domain Network**: Workplace networks where the computer must be part of a domain to communicate.
- **Private Network**: A network that can be discovered; it is a trusted environment just like a home network.
- **Public Network**: Non-discoverable networks meant for untrusted environments like coffee shops or libraries to prevent device discovery.

I inspected each profile individually to verify its configuration:

Domain Network: Confirmed that the Firewall was toggled **On**. I noted that this profile allows for the absolute isolation of incoming traffic if required by corporate security policy.

Private Network & Public Networks: Confirmed that the firewall status was actively toggled **On** for both profiles to ensure baseline protection.

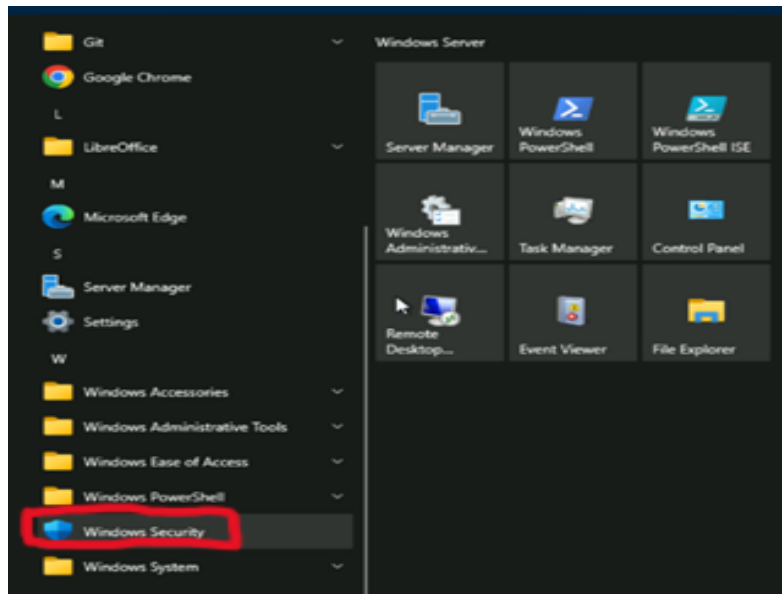


Figure 1: Windows Security

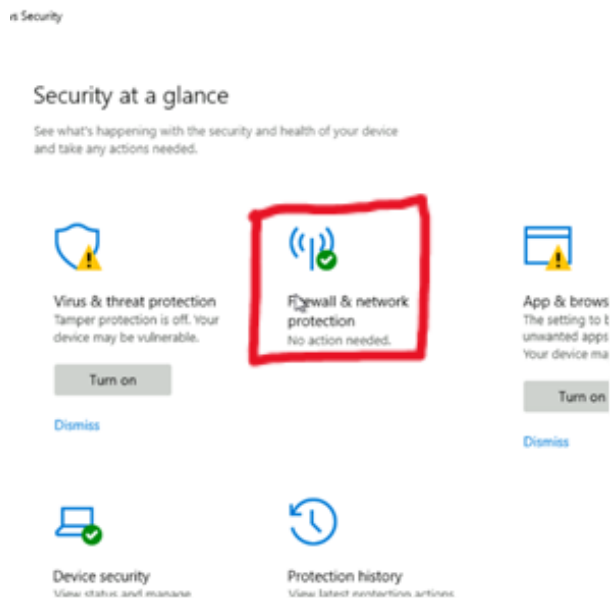


Figure 2: Selected Firewall & Network

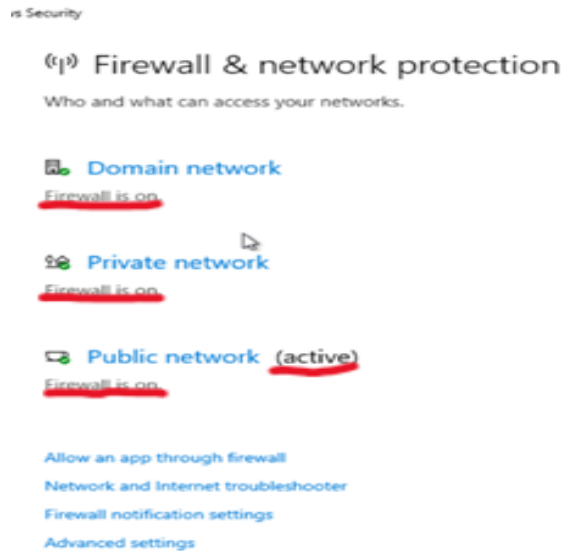


Figure 3: Ensured the firewall is on for various networks

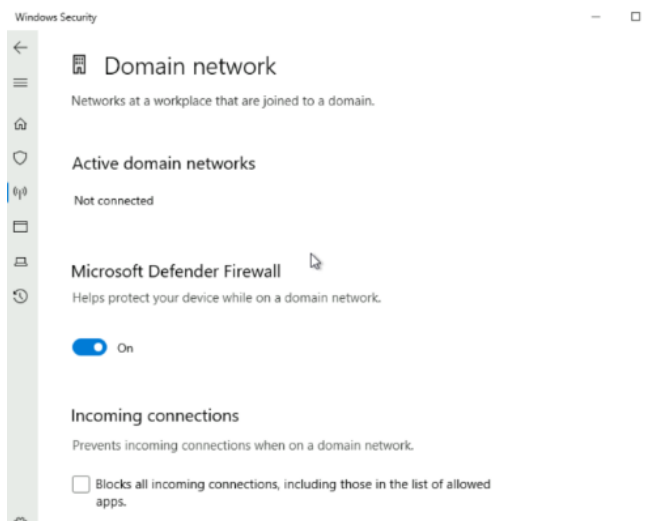


Figure 4: Microsoft Defender Firewall “on”

Step 2: Configuring Inbound Application Rules

Next, I configured Inbound Application Rules. To manage traffic for specific applications, I modified the web browser's permissions by taking the following steps:

- Selected **Allow an app through firewall** from the main menu.
- Located **Mozilla Firefox** (or Google Chrome) in the allowed application list.
- Observed that the initial configuration only allowed Firefox to communicate on the Private network. It was actively blocking it on public networks.
- Selected the **Public** checkbox next to Firefox and clicked **Ok** to apply the changes.

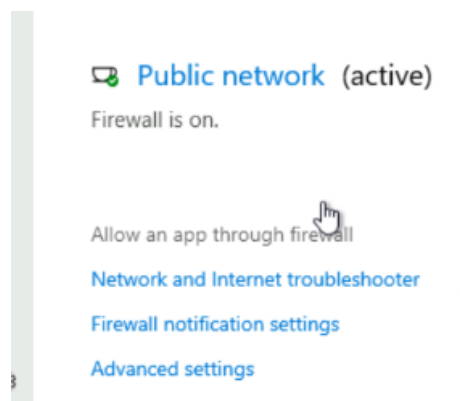


Figure 5: Allow an app

Allow apps to communicate through Windows Defender Firewall

To add, change, or remove allowed apps and ports, click **Change settings**.

What are the risks of allowing an app to communicate?

[Change settings](#)

Allowed apps and features:		
Name	Private	Public
☒ Delivery Optimization	☒	☒
☒ Desktop App Web Viewer	☒	☒
☒ DiagTrack	☒	☒
☒ DIAL protocol server	☒	☐
☐ Distributed Transaction Coordinator	☐	☐
☒ Email and accounts	☒	☒
☐ File and Printer Sharing	☐	☐
☐ File and Printer Sharing over QUIC	☐	☐
☐ File and Printer Sharing over SMBDirect	☐	☐
☒ Firefox (C:\Program Files (x86)\Mozilla Firefox)	☒	☐
☒ Google Chrome	☒	☒
☐ iSCSI Service	☐	☐

Figure 6: Observed Firefox Public Network box unchecked

Allow apps to communicate through Windows Defender Firewall

To add, change, or remove allowed apps and ports, click Change settings.

What are the risks of allowing an app to communicate?

 Change settings

Allowed apps and features:		
Name	Private	Public
☒ Delivery Optimization	☒	☒
☒ Desktop App Web Viewer	☒	☒
☒ DiagTrack	☒	☒
☒ DIAL protocol server	☒	☐
☐ Distributed Transaction Coordinator	☐	☐
☒ Email and accounts	☒	☒
☐ File and Printer Sharing	☐	☐
☐ File and Printer Sharing over QUIC	☐	☐
☐ File and Printer Sharing over SMBDirect	☐	☐
☒ Firefox (C:\Program Files (x86)\Mozilla Firefox)	☒	☒

Figure 7: Checked Firefox Public Network box

Result:

So, I reviewed and configured the Microsoft Defender Firewall settings across various network profiles and managed application-specific traffic permissions. Mozilla Firefox is now authorised to send and receive traffic through the firewall when the device is connected to a public network.

Phase 2: Advanced Traffic Engineering & Hardening

After verifying the baseline profiles, I switched from the simplified consumer interface to configure detailed policies. I opened Windows Defender Firewall with Advanced Security.

For a deep and comprehensive understanding of the platform, I checked the core rule categories displayed in the dashboard:

- **Inbound Rules:** Controlled what traffic is allowed into the host.
- **Outbound Rules:** Here, I restricted what data is allowed to leave the host.
- **Connection Security Rules:** I defined IPsec parameters to encrypt traffic between endpoints.
- **Monitoring:** Tracked real-time traffic logs and active rule enforcement.

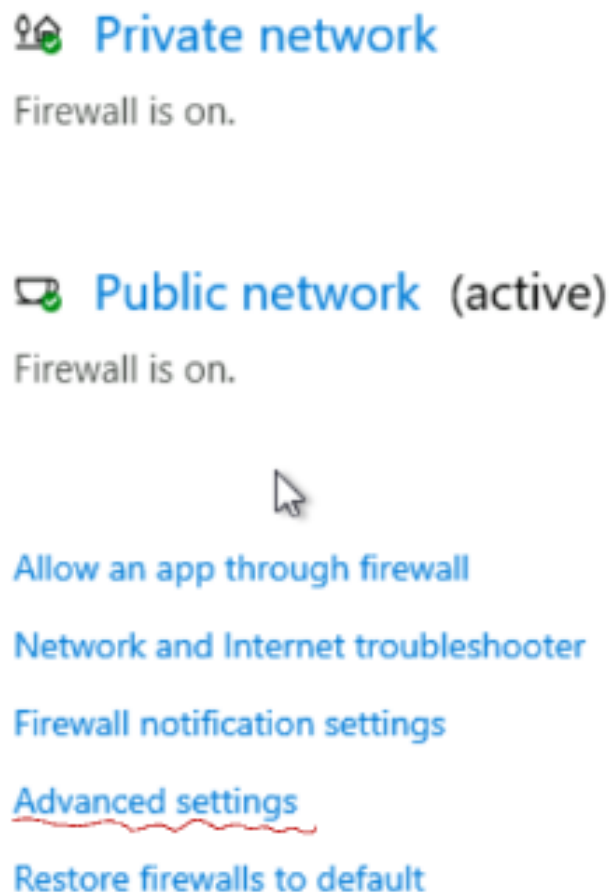


Figure 8: Advanced settings

Scenario A: Suppressing Inbound Remote Desktop (RDP) on Public Networks

Why?

RDP runs on TCP port 3389. If port 3389 is left open on the public network, attackers can scan public Wi-Fi networks to launch a brute-force attack or exploit other unpatched vulnerabilities, such as BlueKeep in corporate devices. Further, blocking RDP would prevent attackers from discovering the device via a network scan (e.g., nmap). Blocking RDP also ensured the device wouldn't allow any remote connections, preventing lateral movement. So, I blocked this port to reduce the endpoint's attack surface by restricting remote management capabilities on untrusted networks.

How?

Rule creation: I navigated to the left pane, clicked "Inbound Rules", and initiated a "New Rule" from the Actions menu.

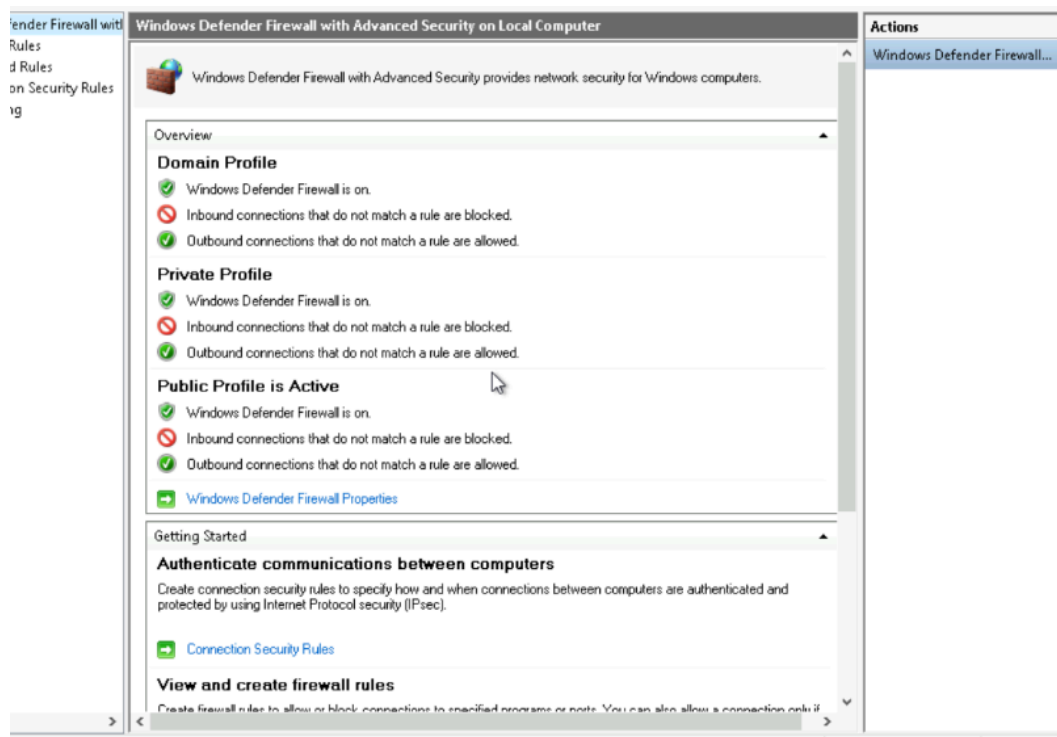


Figure 9: Inbound rules wizard

Criteria Selection: I selected Port as the rule type, specified TCP, and input local port **3389**.

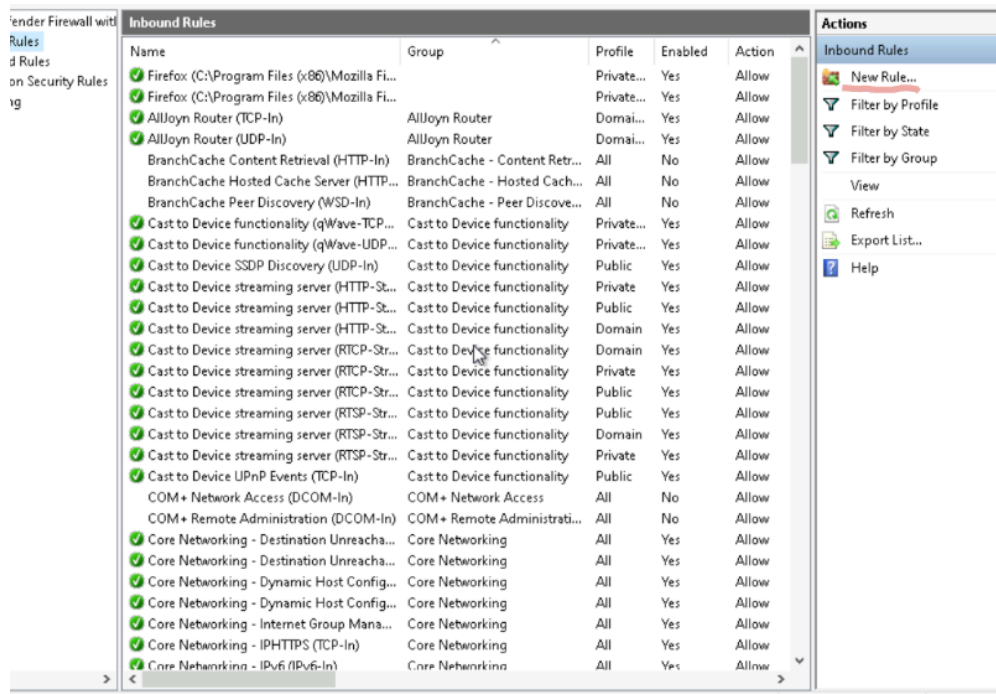


Figure 10: New rules panel

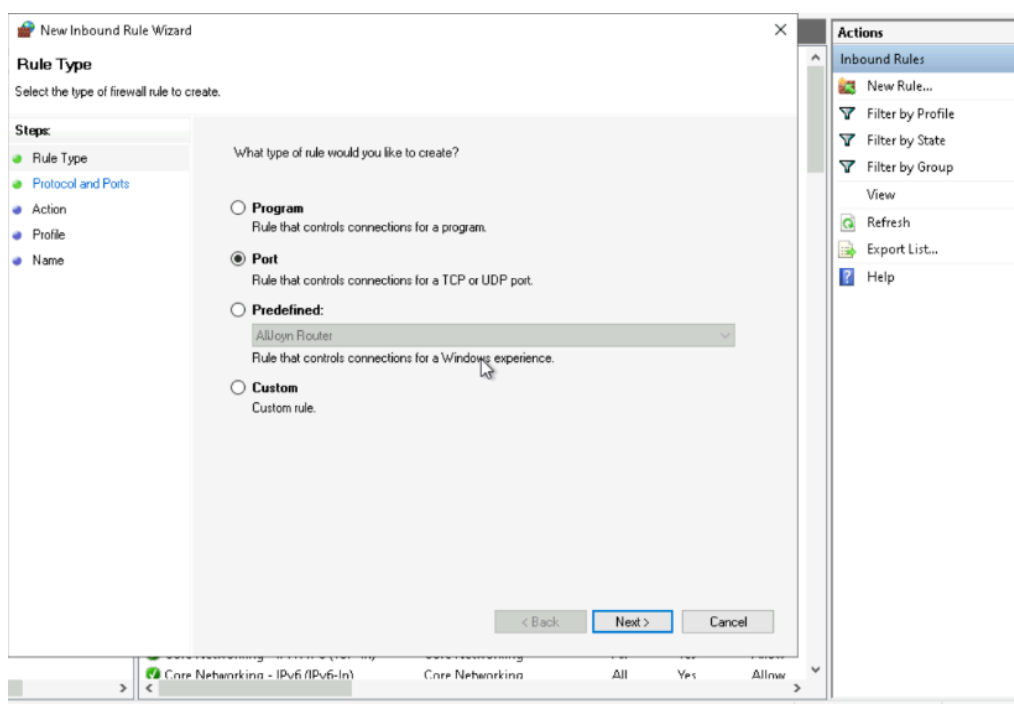


Figure 11: Ports

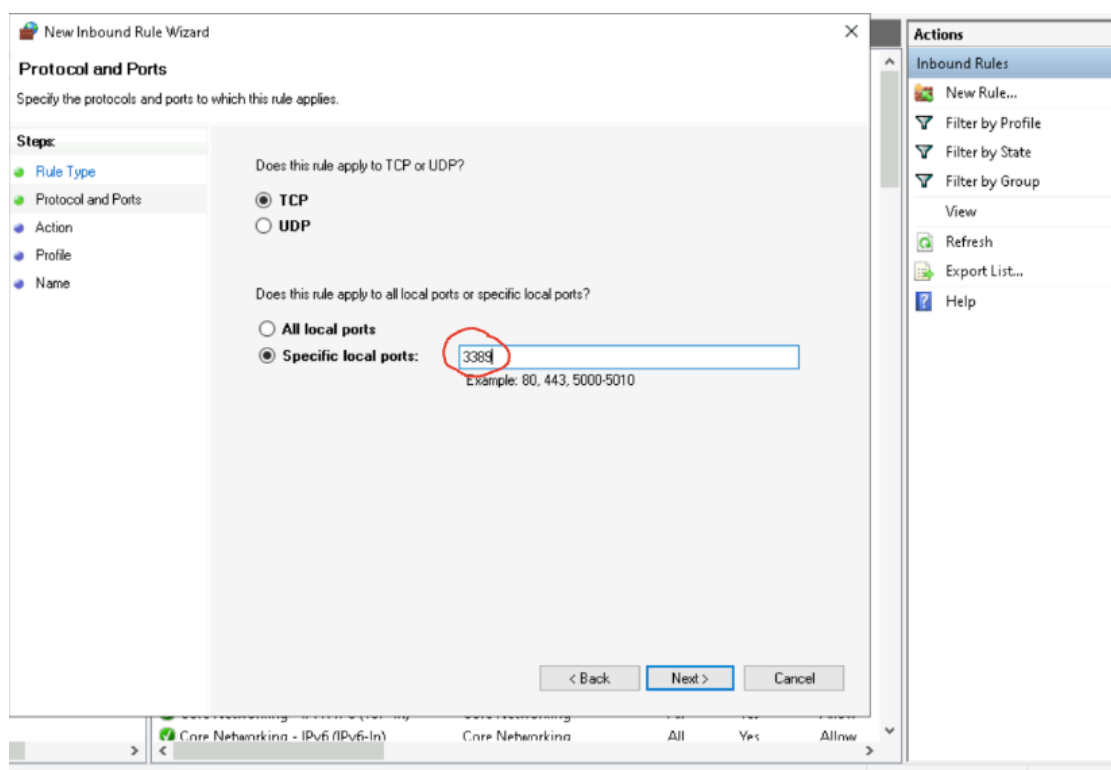


Figure 12: Specific port 3389

Action and Profile Mapping: I selected **Block the connection**. On the profile screen, I customised the scope by checking the **Public** box and unchecking the Domain and Private networks.

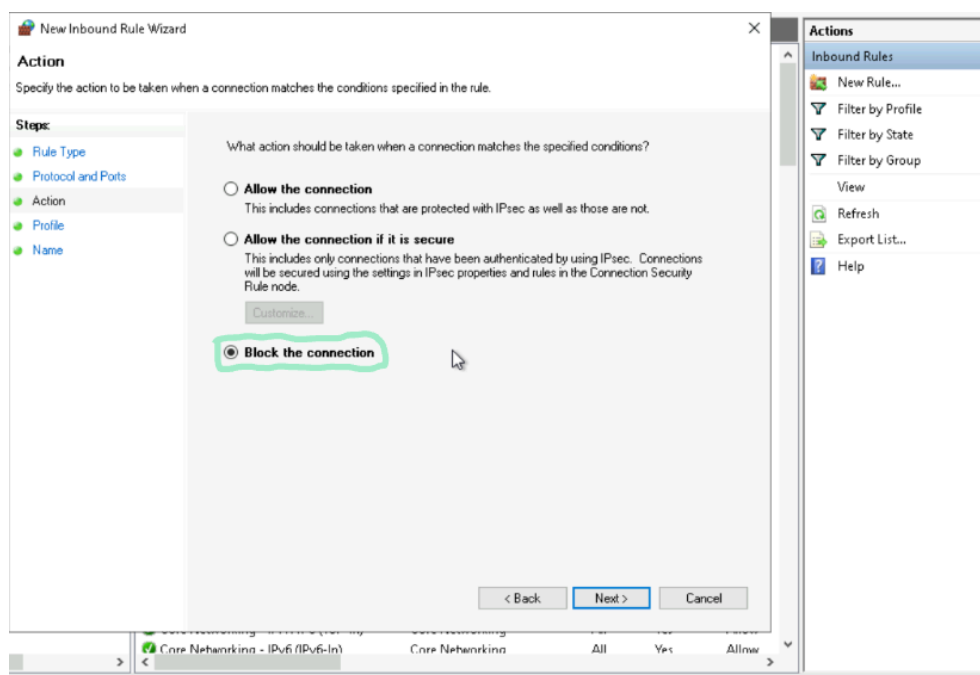


Figure 13: Blocked connection

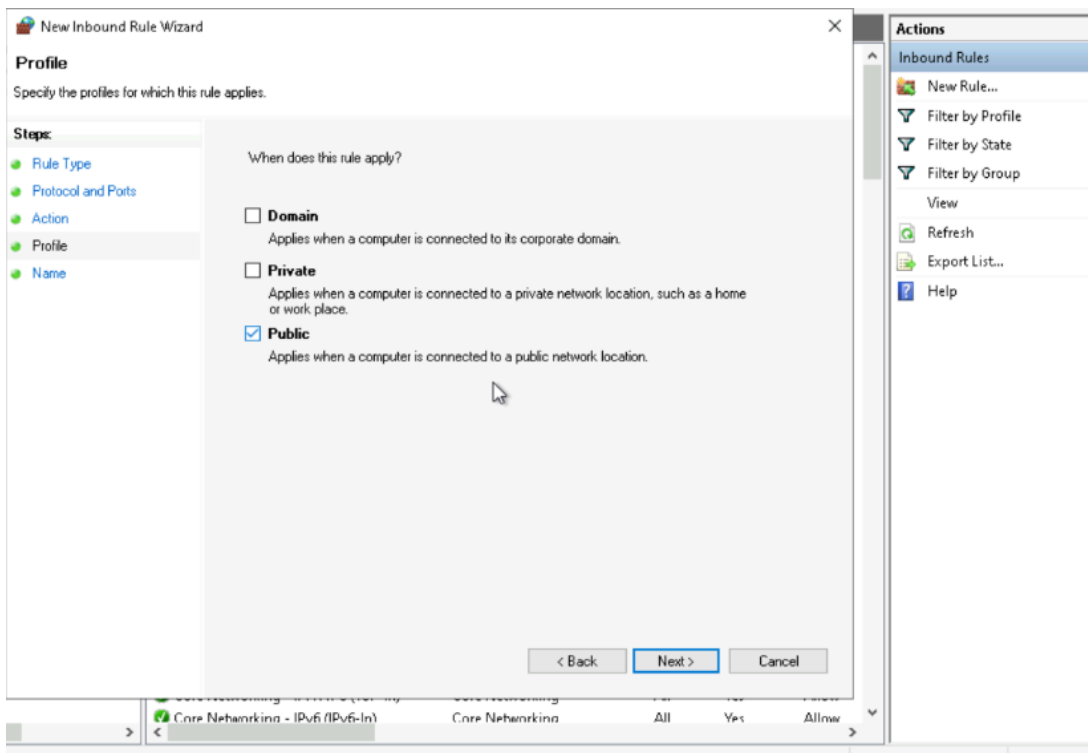


Figure 14: Checked Public network box

Naming: After that, I named the rule “Block Remote Desktop on Public Network” and clicked “Finish”.

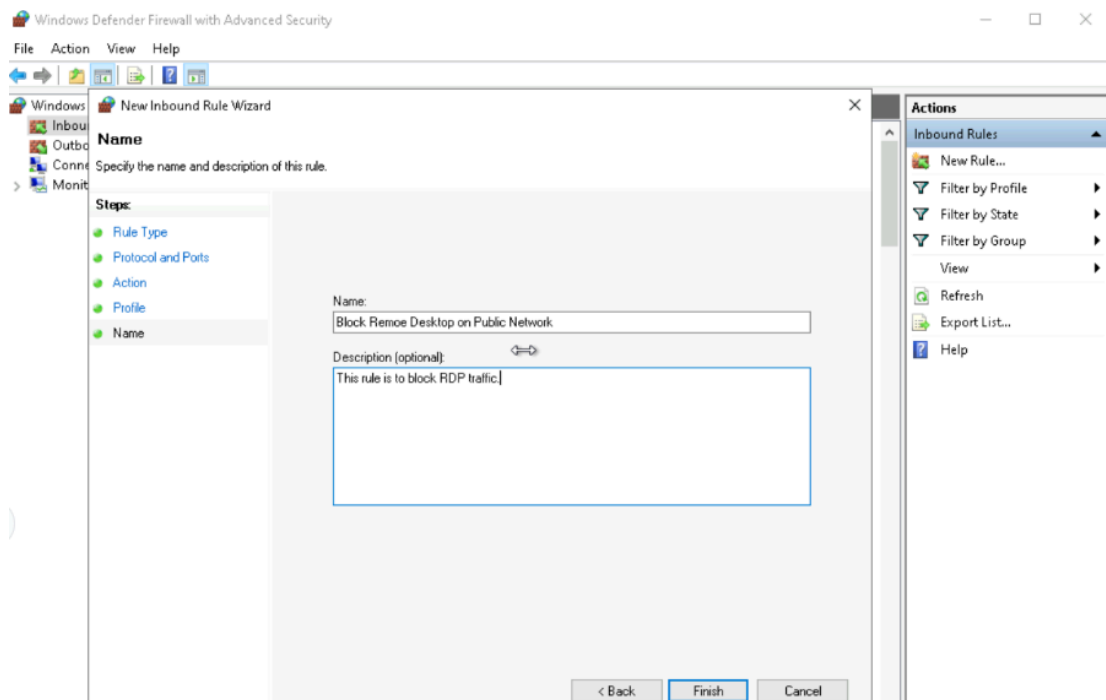


Figure 15: Naming rule

After enabling it, the console displayed a red circle icon next to the rule. Basically, this is confirmation of the active block policy. This action immediately drops unauthorised RDP scans at the firewall level when a user is on public Wi-Fi. This prevents brute-force or password-spraying attacks. When I deployed this rule, the active RDP traffic dropped. Therefore, the session was disconnected.

Scenario B: Preventing Data Exfiltration via Outbound Application Blocking

The objective here was to prevent unauthorised or compromised data from communicating out to the internet.

I have completed scenario B through four steps:

Rule Creation: I navigated to **Outbound Rules** from the left pane and clicked **New Rule**

Program Targeting: Next, I chose **Program** as the rule type and selected the option for this program path. I browsed directly to the absolute path of the executable I wanted to isolate: *C:\Program Files (x86)\Google\Application\chrome.exe*.

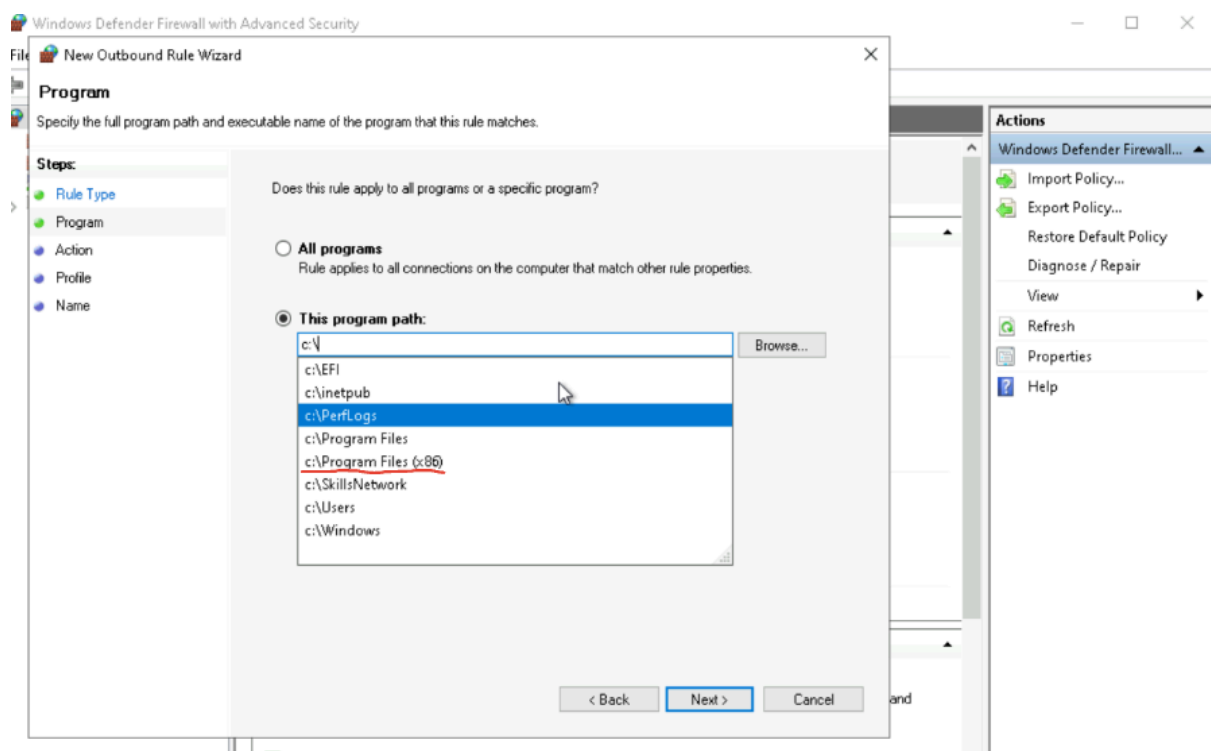


Figure 16: Program path

Action & Scope: After that, I configured the policy to **Block the connection** and applied it across **all network profiles** (Domain, Private, and Public) to ensure total containment.

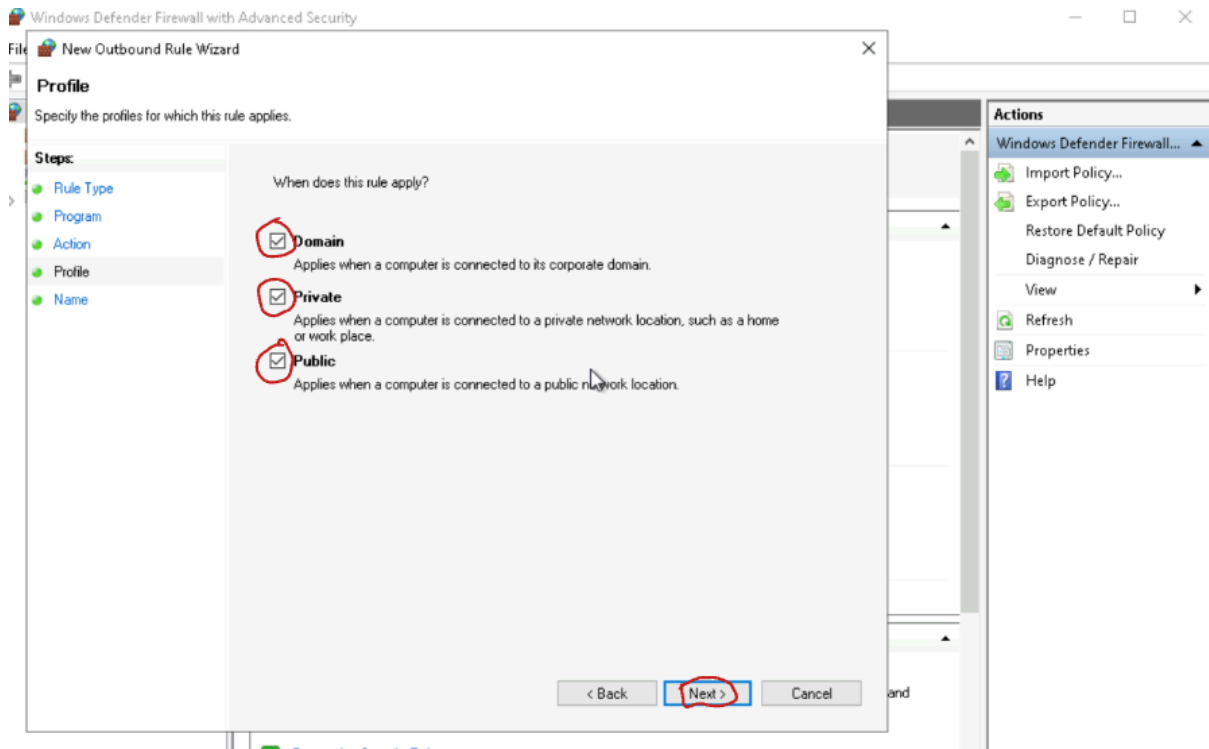


Figure 17: Selected multiple profiles

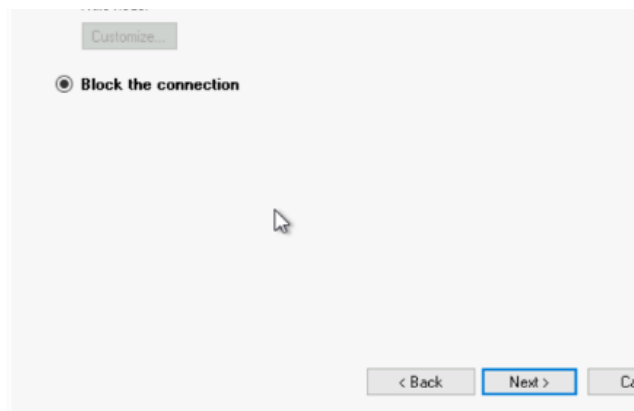


Figure 18: Blocked connection

Naming: Finally, I named the rule “**Block Chrome Internet Access**” and clicked the **Finish** button.

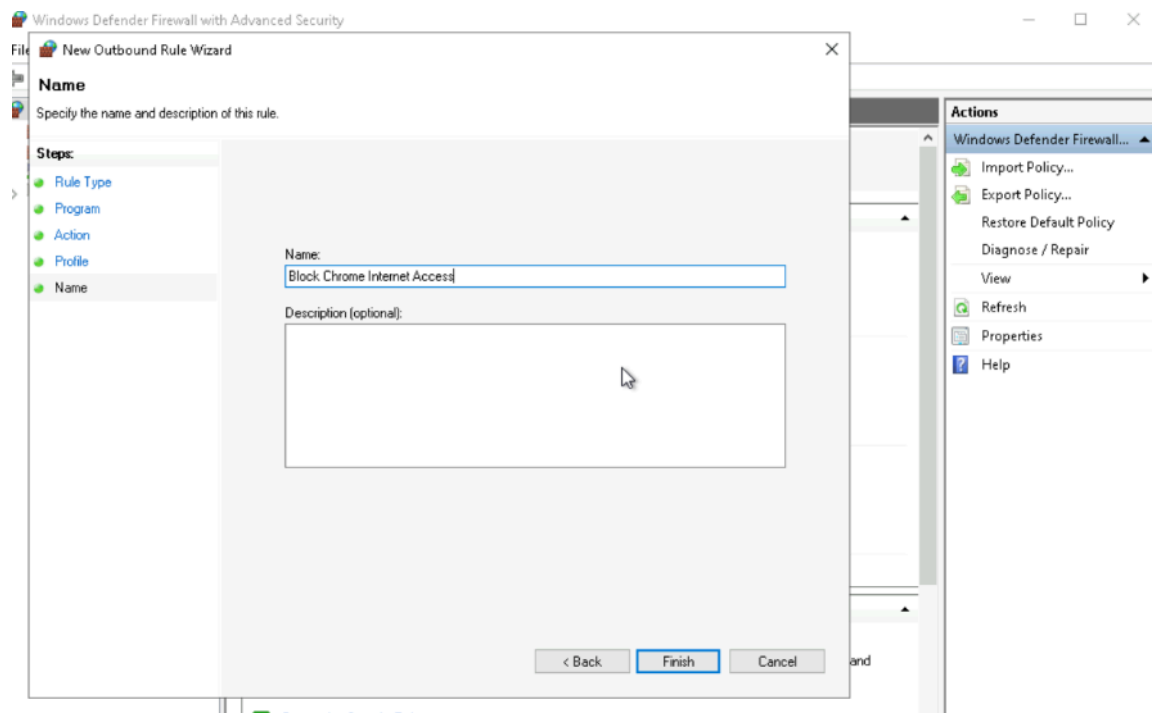


Figure 19: Naming rule

Validation: After the configuration, I navigated to the **Outbound Rules** dashboard and confirmed that the rule is (in a red circle) enabled.

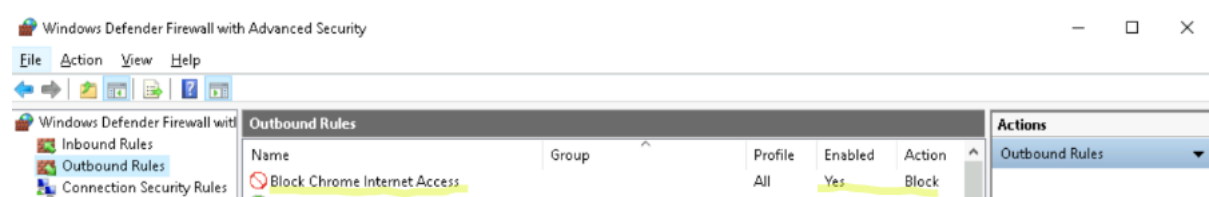


Figure 20: Verified the rule is active

Then I launched Google Chrome and attempted to browse the web. However, the browser immediately failed to load any pages. Next, I went back to the Windows Defender Firewall and disabled the rule. I observed that Chrome instantly regains connectivity. This step successfully validated the host's direct control over software execution.

Scenario C: Mitigating Rogue Web Services (Inbound HTTP Suppression)

The objective of scenario C is to ensure the endpoint does not expose unmanaged web hosting vulnerabilities on untrusted networks. I performed this task in four steps again.

Rule Creation: Within the **Inbound Rules** section, I launched the rule wizard and selected **Port**.

Protocol & Port Configuration: I targeted **TCP** and specified local port 80 (the standard port for cleartext HTTP web traffic).

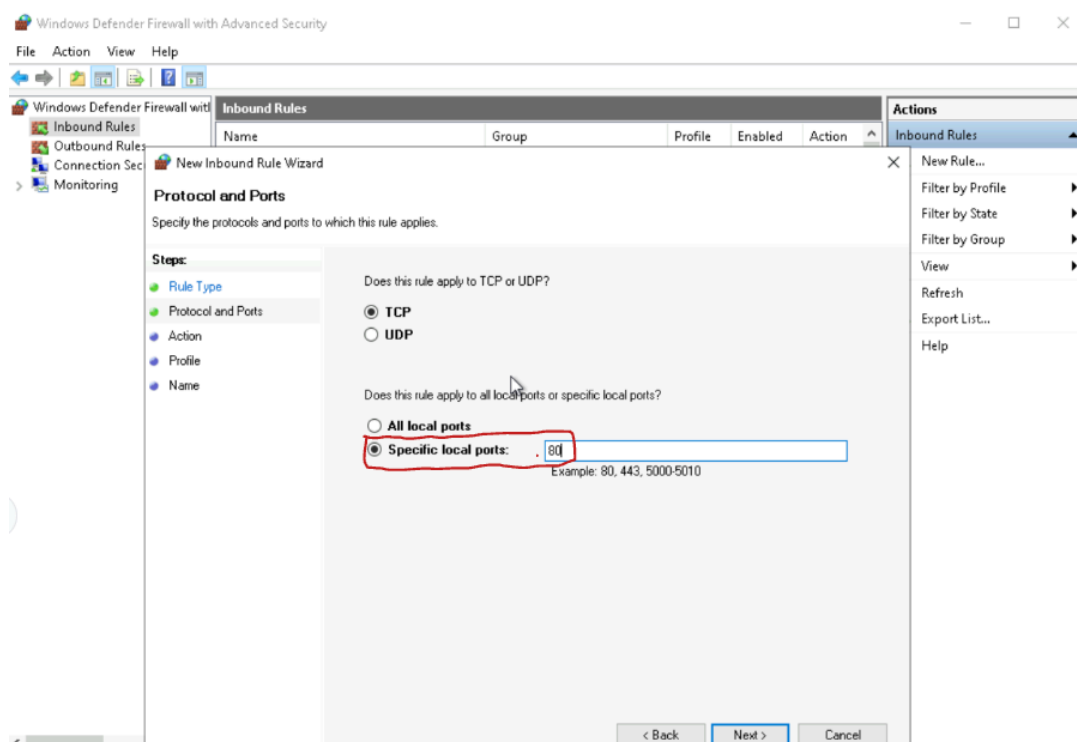


Figure 21: Specified local port 80

Action and Profile Mapping: Next, I chose **Block the connection** to restrict its application to the **Public** profile. Now, it won't impact internal enterprise network configurations.

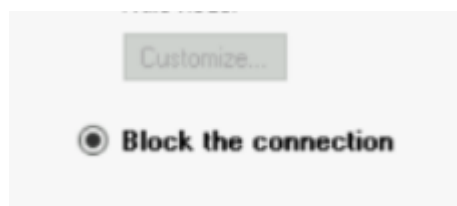


Figure 22: Blocked connection

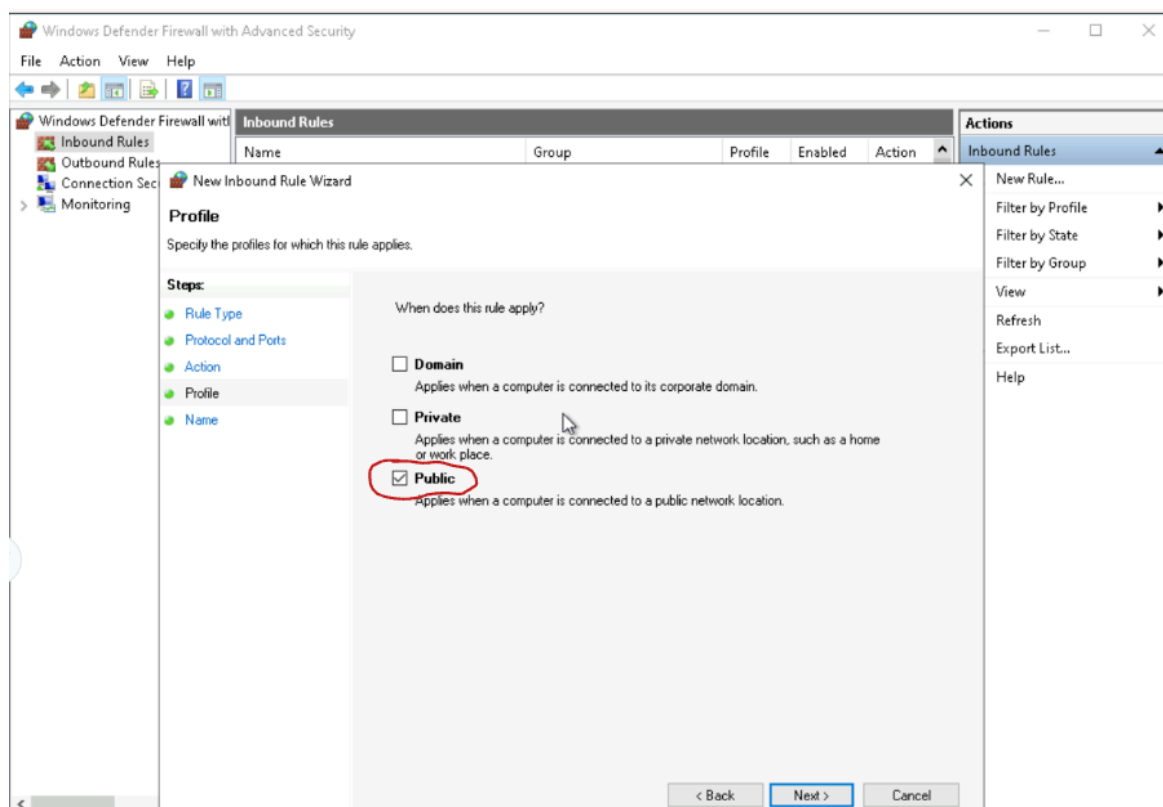


Figure 23: Applied to Public Network

Naming: After the profile mapping, I saved the policy as “Block HTTP on Public Network” and clicked **Finish** to apply.

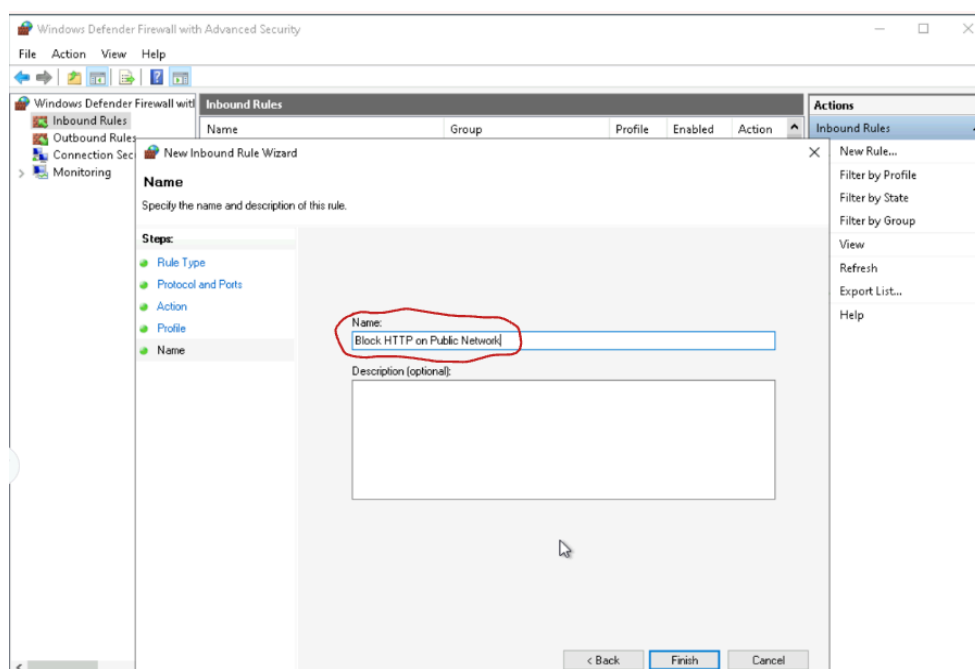


Figure 24: Naming rule

I went back to the main wizard to confirm that the rule is active.

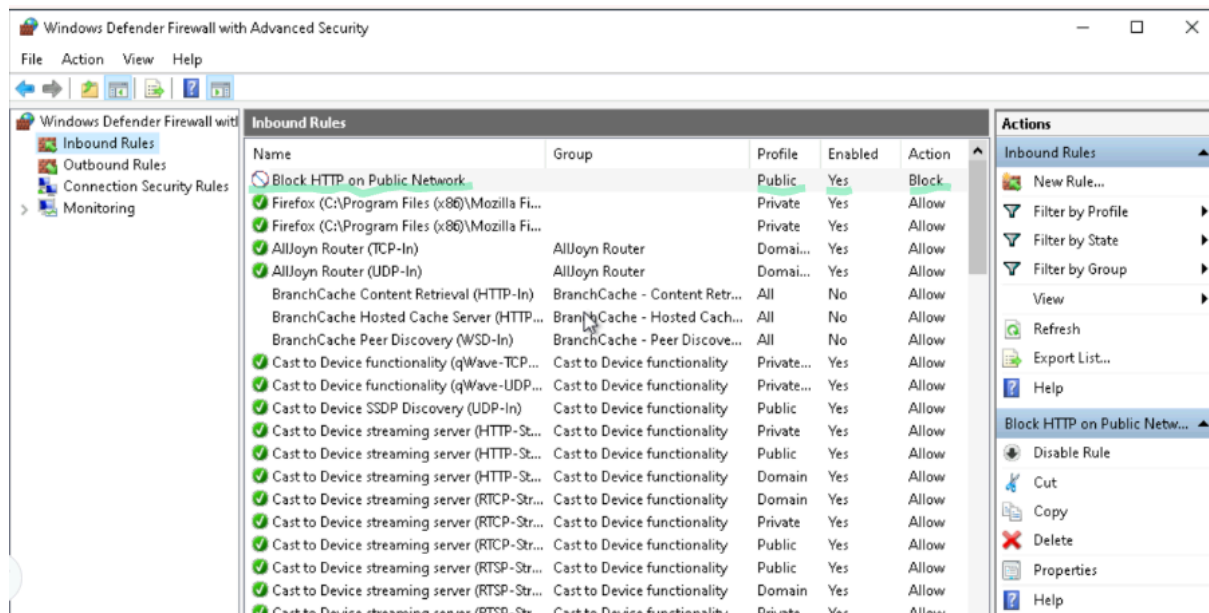


Figure 25: Rule is active

Security Impact: This baseline hardening would ensure that even if a rogue web service or local development tool accidentally runs on the host device, it remains entirely hidden and unreachable to malicious actors scanning the local public network.

Scenario D: Defense-in-Depth Segmentation for Key Management Services (KMS)

Scenario D's objective is to create a secure environment where internal product activation traffic is permitted locally but explicitly denied over public connections.

To prevent future administrative errors from exposing this service, I implemented a dual-rule safety strategy:

Modifying the Default Policy: I navigated the built-in **Key Management Service (TCP-In)** rule. I opened its properties, navigated to the **Advanced** tab, and **unchecked** the **Public** box. Then, I clicked “Apply”. This limited the default allow rule solely to Domain and Private networks.

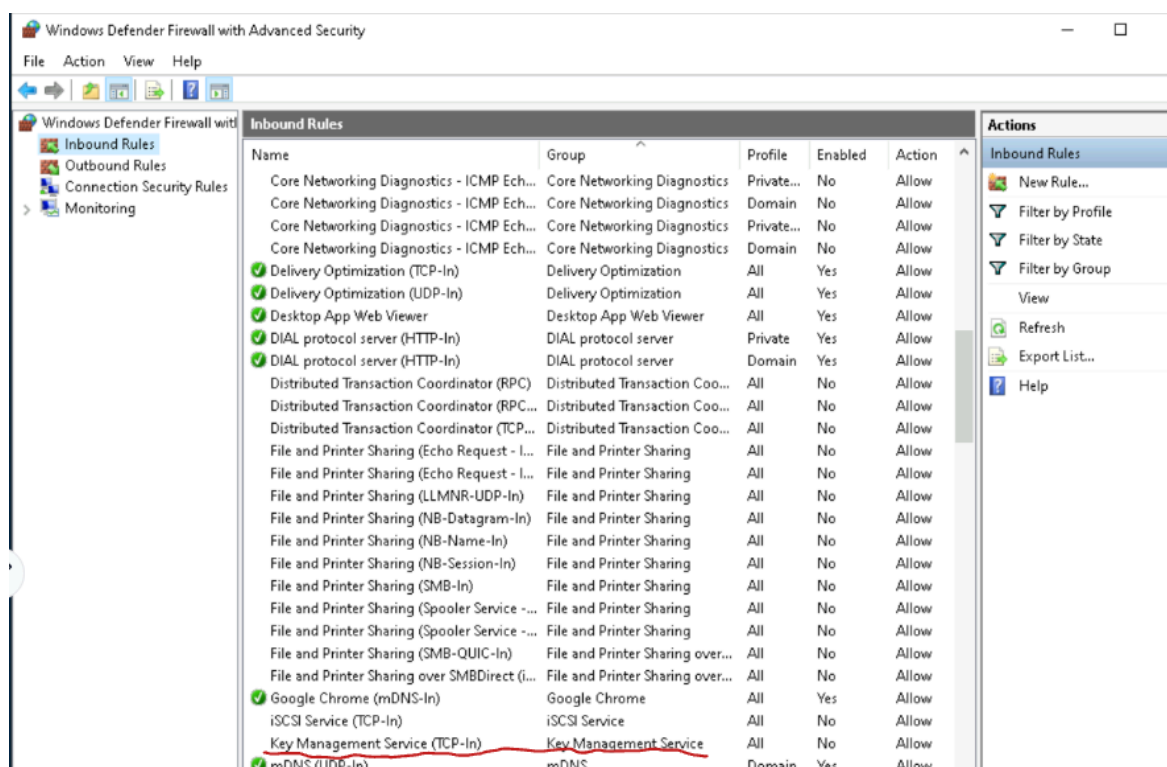


Figure 26: Located Key Management Service (TCP-In)

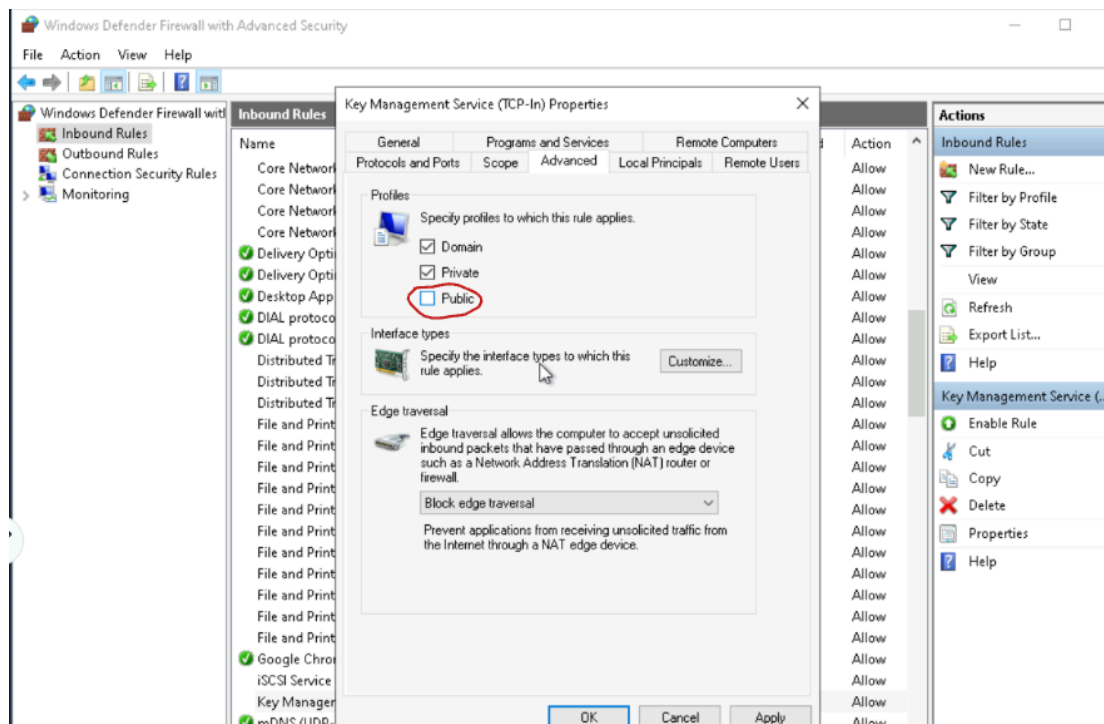


Figure 27: Unchecked Public box

Creating the Explicit Block Backup: To ensure the configuration, I right-clicked the original KMS rule, selected **Copy**, and pasted it to create a duplicate.

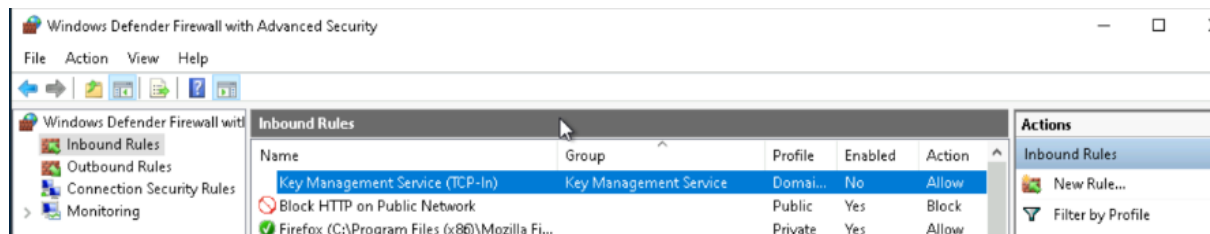


Figure 28: Duplicate KMS

Inverting the Duplicate Rule: I opened the properties of this second rule. Then on the **General** tab, I flipped the action to **Block the connection** (Figure 29).

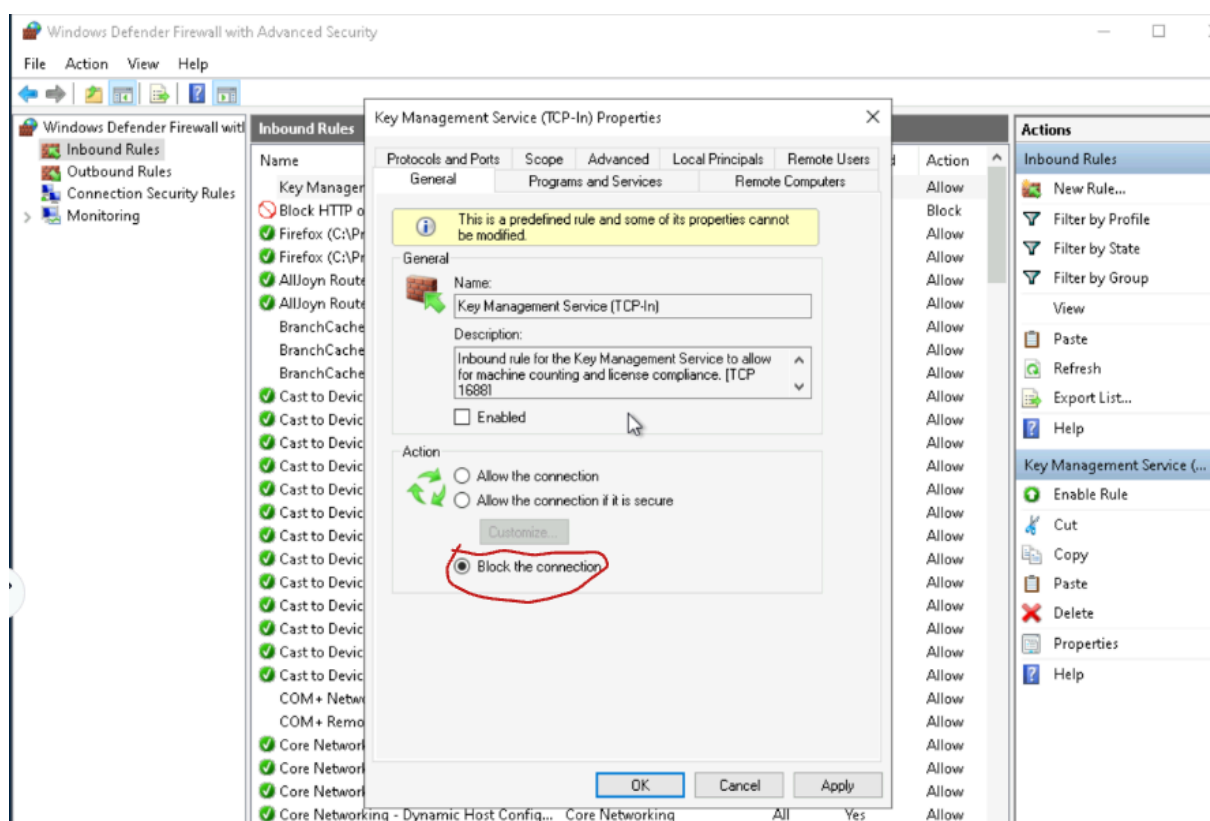
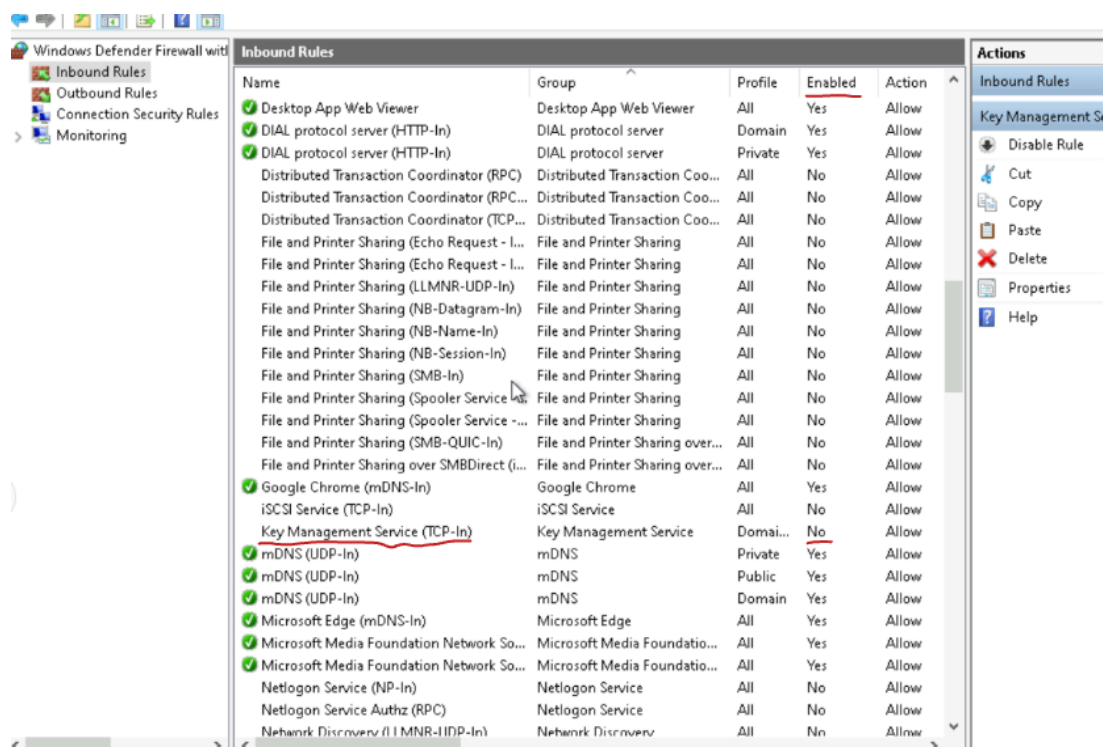
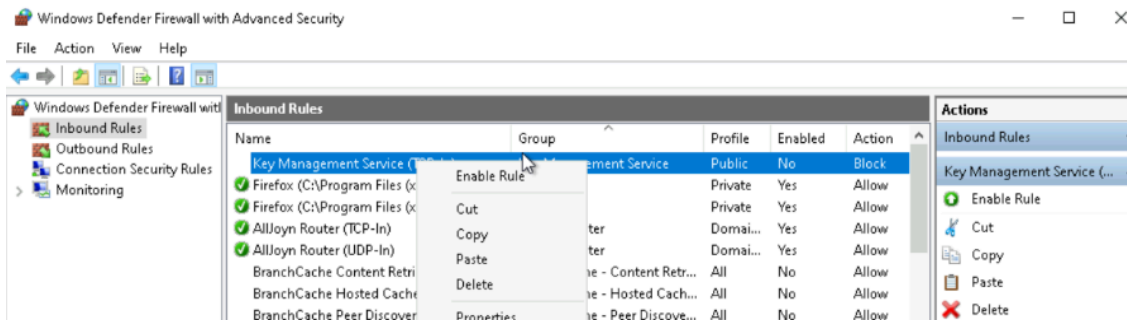
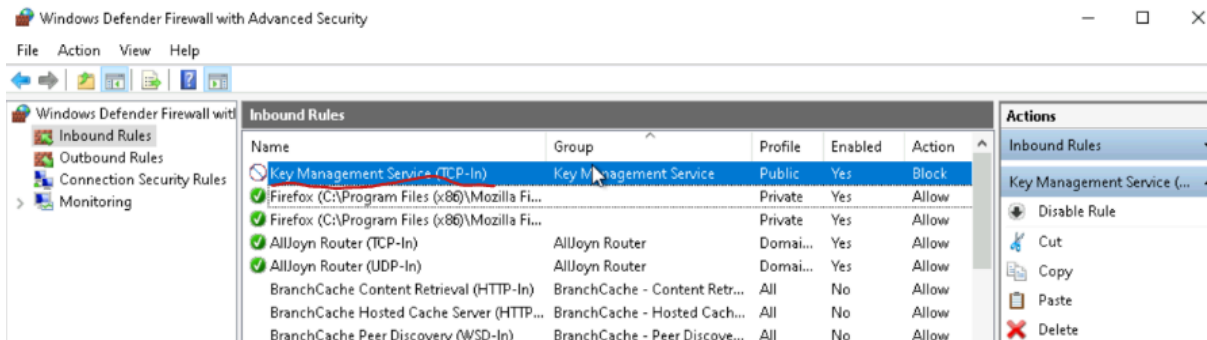
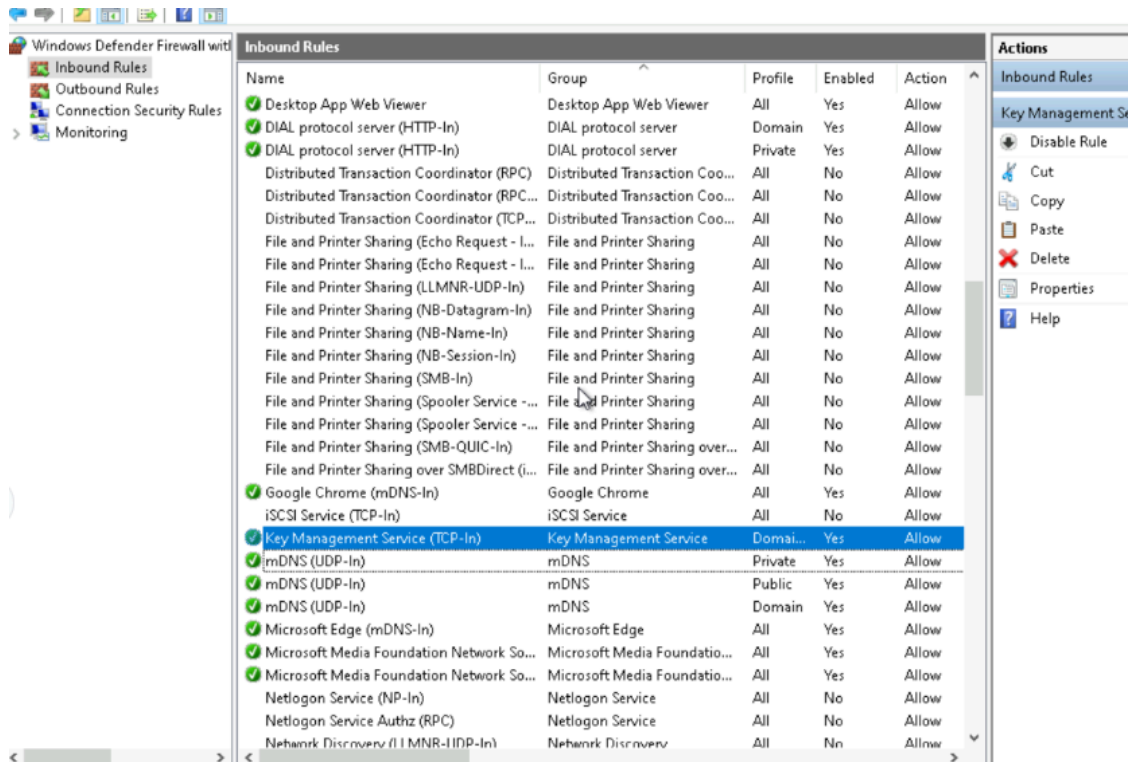


Figure 29

Next, on the **Advanced** tab, I unchecked Domain and Private networks, leaving Public as the only active profile. After that, I right-clicked both rules and selected **Enable Rule**.





Result: The Advanced Security panel updated to show a green checkmark next to the first rule (allowing internal KMS traffic) and a red circle next to the second rule (explicitly blocking public KMS traffic). This defence action guarantees that even if someone accidentally edits the allow rule later, the explicit public block rule will keep the internal enterprise architecture safe.

What went wrong:

When I blocked RDP, the connection got stuck. I had to restart the session a few times to complete the lab.

Conclusion and lessons learned:

Through this lab, I learned how to secure the organisational networks. The lab highlighted the critical role that host-based firewalls play in a **Defend-in-Depth** strategy. Wherever the user travels, hardening endpoints ensures security, whether they are working on the internal corporate domain or remotely on an untrusted public network.

From a SOC and Incident Response perspective, these configurations provide some major advantages, like:

Attack Surface Reduction: Restricting inbound services like RDP (3389) and HTTP (80) on public profiles disables automated scanning and brute-force attempts before they can reach the OS.

Data Exfiltration Mitigation: Controlling outbound application traffic ensures that unauthorised or compromised data can be completely blocked from communicating out to Command and Control (C2) servers.

Network Isolation: Implementing dual-rule strategies ensures internal services like Key Management Services stay strictly internal without risking exposure to the public internet.

Mastering these advanced features of the Windows Defender Firewall settings allowed me to quickly enforce local containment protocols, heavily restricting an attacker's ability to move laterally during an active incident.